So Long, and Thanks for All the Seeds: Attacking GGM-trees in Post-quantum signatures

No Author Given

No Institute Given

Abstract. **Keywords:** GGM tree · Zero-knowledge signatures · Fault attacks · Post-quantum cryptography · Seed compression

1 Introduction

Digital signatures form the backbone of several aspects of modern digital life, ranging from firmware updates and secure boot mechanisms to messaging applications and public-key infrastructures. Their primary goal is to guarantee authenticity: in principle, only the holder of the secret signing key should be able to produce a valid signature for a given message. Additionally, digital signatures provide integrity and non-repudiation, ensuring that a message has not been modified and that the signer cannot later deny having produced the signature. Because of their widespread deployment in security-critical systems, the compromise of a digital signature scheme may have severe consequences, including malicious software updates, device impersonation, and unauthorized access to sensitive communications.

The security of widely used classical signature schemes, such as RSA-based constructions and elliptic-curve-based signatures relying on the Elliptic Curve Discrete Logarithm Problem, would be compromised by sufficiently large quantum computers through algorithms such as Shor's algorithm. This threat has motivated the development of post-quantum cryptography, whose goal is to design cryptographic schemes resistant to both classical and quantum adversaries. Among the most promising candidates are zero-knowledge-based signature schemes relying on Fiat–Shamir transformations of interactive identification protocols. Several recent constructions, including LESS [2], CROSS [3], and MEDS [7], use GGM-tree-like seed compression techniques to reduce signature size by compactly representing large collections of ephemeral randomness. During signing, these schemes selectively reveal only the seeds associated with public rounds while keeping challenge-dependent seeds hidden. Although this mechanism is essential for efficiency, recent works have shown that it also introduces an important fault-attack surface: faults affecting the seed-publication logic may leak hidden seeds together with their corresponding zero-knowledge responses, enabling the recovery of secret information and, ultimately, full key recovery or forgery attacks [11,10].

1.1 Related Work

In the context of fault attacks against post-quantum signature schemes, several works have been proposed in recent years, particularly targeting schemes that rely on GGM-tree-like constructions or related seed-tree compression techniques, such as LESS [2], CROSS [3], and MPC-in-the-Head-based schemes [6,5].

In [10], the authors present a fault attack that exploits the tree-based compression mechanism used in zero-knowledge-based code-based signature schemes. Their primary target is the first version of LESS [2], for which they provide a detailed analysis showing how fault injections in the seed-tree construction can lead to the recovery of secret information and ultimately the signing key. The work further extends the attack analysis to CROSS and briefly discusses how similar ideas could be adapted to MEDS [7], since these schemes rely on related seed-tree compression techniques.

More recently, [11] revisits these attacks in the context of the second version of LESS (LESS-v2). Since LESS-v2 replaces the complete GGM-tree structure of LESS-v1 with an incomplete tree construction, the original attack from [10] no longer applies directly. The authors therefore introduce new fault-assisted forgery attacks targeting the incomplete binary decision-tree generation and seed-publication process. In particular, they identify several fault-injection surfaces within the incomplete tree-generation logic and propose practical instruction-skip-based fault models capable of disclosing hidden ephemeral seeds. A key contribution of the work is the observation that recovering the full signing key is unnecessary for universal forgery; instead, recovering secret-equivalent information, namely the permutations associated with the secret monomial matrices, is sufficient to forge valid signatures.

Still attacking LESS-v2, [9] presents an alternative attack strategy against LESS-v2 that completely avoids targeting the complex tree structure itself. Instead, the attack focuses on the simple `for-loop` responsible for transforming the digest vector b into the binary vector f before the incomplete GGM-tree is generated. Using *iteration-skip* and *loop-abort* fault models, the authors exploit the linear and deterministic nature of this preprocessing stage to recover secret information with high probability. Unlike [11], whose main goal is universal forgery through recovery of secret-equivalent permutations, the attack in [9] aims at full key recovery while also eliminating the need to heuristically target fault locations within the incomplete tree-generation procedure.

Our work unifies these scattered analyses under a single Generic ZK Seed Tree (GZKST) mode, formally proving the correctness and ZK-hiding invariants that all prior attacks implicitly rely on, and extending practical fault attacks to MEDS – a scheme not deep previously studied in this context.

2 Background

2.1 GGM trees

Pseudorandom functions In cryptography, a core challenge is generating randomness that is computationally indistinguishable from true randomness, yet reproducible given a secret key. A **pseudorandom function** (PRF) addresses this need: it is a keyed function that, to any efficient adversary without knowledge of the key, behaves identically to a truly random function. Intuitively, a PRF stretches a short secret key into an exponentially large lookup table of "random-looking" outputs, without ever materialising that table in memory.

PRFs are foundational primitives in symmetric cryptography. They underpin the construction of message authentication codes (MACs), stream ciphers, and key-derivation functions, among many other schemes. As a concrete example, consider AES used in counter mode: given a secret key k and an index i, the output $\mathrm{AES}_k(i)$ is computationally indistinguishable from a uniformly random string, making it a widely deployed candidate PRF in practice.

A central construction of PRFs from one-way functions is the **Goldreich–Goldwasser–Micali (GGM) tree** [8], which builds a PRF by iteratively applying a pseudorandom generator along the branches of a binary tree indexed by the input bits. GGM trees are the primary PRF construction studied in this work.

Definition 1 (GGM Tree). *Let* $\mathsf{G} : \{0,1\}^\lambda \to \{0,1\}^{2\lambda}$ *be a length-doubling pseudorandom generator, decomposed as* $\mathsf{G}(x) = (\mathsf{G}_0(x), \mathsf{G}_1(x))$*, where* G_0 *and* G_1 *each output* λ *bits. Given a root seed* $\mathsf{seed}_r \in \{0,1\}^\lambda$ *and a depth* d*, the* GGM tree *is a complete binary tree of depth* d *in which:*

- *the root holds* seed_r*;*
- *each internal node* v *storing seed* s *generates its left child as* $\mathsf{G}_0(s)$ *and its right child as* $\mathsf{G}_1(s)$*;*
- *the* 2^d *leaves are the* G *outputs at depth* d*.*

The crucial property exploited by ZK signature schemes is the *punctured-PRF property*: given all leaf seeds *except* leaf i, it is computationally infeasible to recover the seed at leaf i, provided G is a secure pseudorandom generator. This makes the GGM tree suitable for hiding one or more seeds while revealing the rest, as shown in Figure 1.

Incomplete GGM *trees.* When the number of required leaves t is not a power of 2, a scheme may use an *incomplete* tree: leaves appear at multiple levels, and internal nodes near the bottom may have fewer than two children or act as leaves themselves. This reduces memory and computation, but the additional index-tracking logic introduces new fault surfaces, as detailed in Section 4.

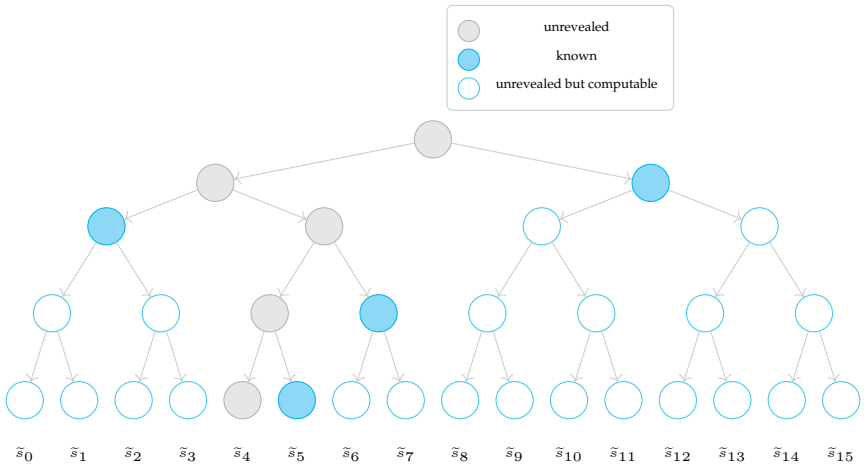


Fig. 1: Receiver's GGM tree of depth 4. Given only the blue nodes and the PRNG function, the receiver can retrieve all leaves except x_4.

2.2 Attack targets

This section provides an overview of the signature schemes, to which we apply our generalized model and against which we carry out our attacks.

LESS-v2 LESS (Linear Equivalence Signature Scheme) [2] uses a Fiat–Shamir transformation over a 3-pass Sigma protocol whose security rests on the Canonical Form Linear Equivalence Problem (CF-LEP). During signing, the signer must generate t independent ephemeral monomial matrices $\mathbf{Q}_{e_0}, \ldots, \mathbf{Q}_{e_{t-1}} \in \mathrm{Mon}_n(q)$.

Role of the GGM tree in signing and verification. Each leaf $\widetilde{s}_i$, for $0 \leq i < t$, is an ephemeral seed of length λ bits from which the i-th monomial map $\mathbf{Q}_{e_i} \in \mathrm{Mon}_n(q)$ is derived by expanding $\widetilde{s}_i$ through a PRNG. The tree thus compactly encodes all t independent ephemeral matrices required by the iterated Sigma protocol from a single master seed $\mathsf{mseed} \in \{0,1\}^\lambda$.

During signing, the Fiat–Shamir challenge vector $\mathbf{b} = (b[0], \ldots, b[t-1]) \in \mathcal{S}_{t,w}$ partitions the rounds into two sets: the w rounds with $b[i] \neq 0$, for which the signer publishes $\mathsf{CosetRep}(\pi_{b[i]} \circ \tilde{\pi}_i^*)$ and must *conceal* $\widetilde{s}_i$, and the remaining $t-w$ rounds, for which $\widetilde{s}_i$ itself is disclosed so that the verifier can re-derive $\mathbf{Q}_{e_i}$ and re-compute the commitment $\mathrm{CF}(A_i)$.

On the verifier side, each received node is expanded downward by the PRNG until all required leaves are reconstructed; the hidden leaves, corresponding to rounds with $b[i] \neq 0$, are never reachable from any published node, so the ephemeral seeds $\widetilde{s}_i$ remain protected.

Secret-extraction opportunity. For every round i with $b[i] \neq 0$, the signer publishes $\mathsf{CosetRep}(\pi_{b[i]} \circ \tilde{\pi}_i^*)$ while keeping $\widetilde{s}_i$ hidden. If an attacker obtains both values simultaneously, Theorem 2 of [11] shows that the secret permutation $\pi_{b[i]}$ is recoverable. The flag tree is the *sole enforcement mechanism* that prevents this dual revelation. Table 1 lists the LESS-v2 parameter sets.

CROSS

Context. CROSS (Codes and Restricted Objects Signature Scheme) [3] is based on the Restricted Syndrome Decoding Problem (R-SDP) [4] and applies the Fiat–Shamir transform to a 5-pass (q_2-identification) protocol. Like MEDS, CROSS uses a *binary* fixed-weight challenge vector $\mathbf{c} = (c_0, \ldots, c_{t-1}) \in \{0,1\}^t$ of weight w, so the hidden and revealed index sets are $\mathcal{H} = \{i : c_i = 1\}$ and $\mathcal{R} = \{i : c_i = 0\}$, identical in structure to MEDS.

Role of the seed tree in signing and verification. Each leaf Seed_i, for $0 \leq i < t$, is a λ-bit ephemeral seed from which the pair of randomness vectors $(\mathbf{e}'_i,\ \mathbf{u}'_i) \in G \times \mathbb{F}_p^n$ is derived via $\mathsf{CSPRNG}(\mathsf{Seed}_i)$. These vectors are the sole secret material for round i: from them the signer computes the linear transitive map $\mathbf{v}_i = \mathbf{e} \star (\mathbf{e}'_i)^{-1} \in G$ and the two commitments $\mathsf{cmt}_0[i] = \mathrm{Hash}((\mathbf{v}_i \star \mathbf{u}'_i)\mathbf{H}^\top \mid \mathbf{v}_i)$ and $\mathsf{cmt}_1[i] = \mathrm{Hash}(\mathbf{u}'_i \mid \mathbf{e}'_i)$. The seed tree thus compactly encodes all t pairs of ephemeral vectors required by the iterated Sigma protocol from a single master seed.

During signing, the Fiat–Shamir second challenge vector $\mathsf{chall}_2 \in \mathcal{B}(t, w)$ of fixed Hamming weight w partitions the rounds into two sets: the w rounds with $\mathsf{chall}_2[i] = 1$, for which the signer publishes the explicit response $\mathsf{resp}[i] = (\mathbf{y}_i, \mathbf{v}_i)$ and must *conceal* Seed_i, and the remaining $t - w$ rounds with $\mathsf{chall}_2[i] = 0$, for which Seed_i itself is disclosed so that the verifier can re-expand $(\mathbf{e}'_i, \mathbf{u}'_i)$, recompute $\mathbf{y}_i = \mathbf{u}'_i + \mathsf{chall}_1[i] \cdot \mathbf{e}'_i$, and recover $\mathsf{cmt}_1[i]$.

Additionally, CROSS uses a parallel *Merkle tree* over the t commitments $\mathsf{cmt}_0[i]$ to avoid transmitting all t digests explicitly: for the w rounds where $\mathsf{chall}_2[i] = 1$, the corresponding $\mathsf{cmt}_0[i]$ values cannot be recomputed by the verifier (since Seed_i is hidden), and are instead authenticated via a Merkle proof of size $O(w \log t)$.

On the verifier side, the received seed-path nodes are expanded downward to reconstruct all unrevealed leaf seeds; the w hidden seeds, corresponding to rounds with $\mathsf{chall}_2[i] = 1$, are provably unreachable from any published node, ensuring that the ephemeral vectors $(\mathbf{e}'_i, \mathbf{u}'_i)$ — and hence the secret key $\mathbf{e}$ — remain protected.

Secret-extraction opportunity. For round i with $c_i = 1$, the signer reveals a ZK response encoding the relationship between $\widetilde{s}_i$ and the long-term secret R-SDP solution. If an attacker obtains both the response and $\widetilde{s}_i$, the secret is recoverable by the linear-algebra argument of ZKFault [10], which demonstrated full key recovery from a single fault for all CROSS parameter sets (see Table 1).

MEDS MEDS (Matrix Equivalence Digital Signature) [7] is a code-based signature scheme whose security rests on the Matrix Code Equivalence Problem (MCEP). Like LESS, it applies the Fiat–Shamir transform to a 3-pass Sigma protocol between a prover holding a secret equivalence $(A, B) \in \mathrm{GL}_k(q) \times \mathrm{GL}_n(q)$ and a verifier.

Role of the seed tree in signing and verification. Each leaf σ_i, for $0 \le i < t$, is an ephemeral seed of length $\ell_{\text{tree\_seed}}$ bytes from which the i-th pair of ephemeral invertible matrices $(\tilde{A}_i, \tilde{B}_i) \in \mathrm{GL}_m(q) \times \mathrm{GL}_n(q)$ is sampled via a PRNG. The commitment for round i is then $h_i = H\big(\mathrm{SF}(\pi_{\tilde{A}_i,\tilde{B}_i}(G_0))\big)$, so the seed tree compactly encodes all t ephemeral matrix pairs required by the iterated Sigma protocol from a single master seed $\rho_{0,0} \in \mathcal{B}^{\ell_{\text{tree\_seed}}}$.

During signing, the Fiat–Shamir challenge string $\mathbf{h} = (h_0, \ldots, h_{t-1}) \in \{0, \ldots, s-1\}^t$ of fixed weight w partitions the rounds into two sets: the w rounds with $h_i \neq 0$, for which the signer publishes the coset representative $(\mu_i, \nu_i) = (A\tilde{A}_i^{-1},\ B^{-1}\tilde{B}_i)$ and must *conceal* σ_i, and the remaining $t-w$ rounds with $h_i = 0$, for which σ_i is disclosed so that the verifier can re-expand $(\tilde{A}_i, \tilde{B}_i)$ and recover the commitment h_i.

On the verifier side, the required leaf seeds are recovered; the w hidden seeds, corresponding to rounds with $h_i \neq 0$, are provably unreachable from any published node.

Since we are going to show in §5 an application of the attack in the reference implementation of MEDS, we are going to explain better the algorithms of key generation, signing and verification. Table 1 shows the parameters for MEDS [7].

Key Generation. Algorithm 1 is the simplification of the key generation. It begins by sampling a master secret seed δ and immediately deriving two sub-seeds via XOF: a *public* seed σ_{G_0}, used to expand the base systematic matrix $G_0 \in \mathbb{F}_q^{k \times mn}$, and a *private* chaining seed σ that is used in the rest of the process. For each of the $s-1$ public keys, the algorithm performs a *compressed key-generation* step: a secret invertible matrix $T_i \in \mathrm{GL}_k(q)$ twists G_0 into $G_0' = T_i G_0$, and a structured linear system (the Solve step, made efficient by fixing $P_0 = I_m$ and $P_1 = U_m$) recovers the equivalence pair $(A_i, B_i) \in \mathrm{GL}_m(q) \times \mathrm{GL}_n(q)$ satisfying $G_i = \mathsf{SF}(\pi_{A_i,B_i}(G_0))$. The public key stores only the seed σ_{G_0} and the compressed matrices $G_1, \ldots, G_{s-1}$; the secret key retains δ together with the stored inverses A_i^{-1}, B_i^{-1}—the sole secret material consumed at signing time.

Signing. Algorithm 2 is the simplification of the signing algorithm. A fresh master seed δ is drawn and expanded into a tree-root seed ρ and a salt α; the seed tree $\mathsf{SeedTree}_t(\rho, \alpha)$ then materialises all t leaf seeds $\sigma_0, \ldots, \sigma_{t-1}$ at once. For every round i, a pair of ephemeral matrices $(\tilde{A}_i, \tilde{B}_i) \in \mathrm{GL}_m(q) \times \mathrm{GL}_n(q)$ is expanded from σ_i (together with α and an address for domain separation), and the commitment $\tilde{G}_i = \mathsf{SF}(\pi_{\tilde{A}_i,\tilde{B}_i}(G_0))$ is computed. Hashing all non-systematic parts of the $\tilde{G}_i$ alongside the message yields the digest d, which is parsed into

Table 1: Cryptographic parameters for LESS-v2 [2], MEDS [7], and CROSS [3] ($\star$: MEDS uses $n{=}m{=}k$; CROSS uses $q{=}2$ implicitly via $\mathbb{F}_p$).

Scheme	Parameter set	t	w	s	Sec.	Extra
LESS-v2	LESS-252-45	45	34	8	I	$n{=}252$, $k{=}126$, $q{=}127$
	LESS-252-68	68	42	4	I	$n{=}252$, $k{=}126$, $q{=}127$
	LESS-252-192	192	36	2	I	$n{=}252$, $k{=}126$, $q{=}127$
	LESS-400-102	102	61	4	III	$n{=}400$, $k{=}200$, $q{=}127$
	LESS-400-220	220	68	2	III	$n{=}400$, $k{=}200$, $q{=}127$
	LESS-548-137	137	79	4	V	$n{=}548$, $k{=}274$, $q{=}127$
	LESS-548-345	345	75	2	V	$n{=}548$, $k{=}274$, $q{=}127$
MEDS*	MEDS-9923	1152	14	4	I	$n{=}14$, $q{=}4093$
	MEDS-13220	192	20	5	I	$n{=}14$, $q{=}4093$
	MEDS-41711	608	26	4	III	$n{=}22$, $q{=}4093$
	MEDS-69497	160	36	5	III	$n{=}22$, $q{=}4093$
	MEDS-134180	192	52	5	V	$n{=}30$, $q{=}2039$
	MEDS-167717	112	66	6	V	$n{=}30$, $q{=}2039$
CROSS	CROSS-R-SDP-1	163	60	2	I	$p{=}127$, $n{=}68$, $k{=}8$
	CROSS-R-SDP-3	200	75	2	III	$p{=}127$, $n{=}80$, $k{=}9$
	CROSS-R-SDP-5	250	94	2	V	$p{=}127$, $n{=}90$, $k{=}9$
	CROSS-R-SDPe-1	55	25	2	I	$p{=}509$, $n{=}60$, $k{=}6$
	CROSS-R-SDPe-3	75	34	2	III	$p{=}509$, $n{=}80$, $k{=}8$
	CROSS-R-SDPe-5	90	40	2	V	$p{=}509$, $n{=}90$, $k{=}8$

the fixed-weight challenge vector $(h_0, \ldots, h_{t-1})$. For each *non-zero* challenge h_i, the signer releases the coset representative $(\mu_i, \nu_i) = (\tilde{A}_i A_{h_i}^{-1},\ B_{h_i}^{-1}\tilde{B}_i)$; the remaining seeds are transmitted compactly via a seed-tree path p.

Verification. Algorithm 3 is the simplification of the verification algorithm. The verifier first reconstructs the public matrices $G_0, G_1, \ldots, G_{s-1}$ from the public key and parses the signed message to recover the path p, the digest d, the salt α, and the w response pairs. The challenge vector $(h_0, \ldots, h_{t-1})$ is recomputed from d, and $\mathsf{PathToSeedTree}_t$ recovers the $t - w$ disclosed leaf seeds from p. For each round i, one of two paths is taken: if $h_i > 0$, the verifier applies the published coset representative (μ_i, ν_i) to the corresponding public matrix G_{h_i} and checks invertibility before computing $\hat{G}_i = \mathsf{SF}(\pi_{\mu_i,\nu_i}(G_{h_i}))$; if $h_i = 0$, the verifier re-expands $(\hat{A}_i, \hat{B}_i)$ from the recovered seed and computes $\hat{G}_i = \mathsf{SF}(\pi_{\hat{A}_i,\hat{B}_i}(G_0))$. Finally, the recomputed digest d' is compared to d; equality confirms the signature.

Secret-extraction opportunity. For round i with $h_i = 1$, the signer reveals its ZK response. If an attacker obtains both the response and $\widetilde{s}_i$, the secret equivalence pair (A, B) is recoverable by linear algebra. The Reference Tree is the sole enforcement mechanism preventing this dual revelation.

Algorithm 1: MEDS Key Generation (simplified)

Input: —
Output: Public key pk, secret key sk

$\delta \xleftarrow{\$} \mathcal{B}^{\ell_{\text{sec\_seed}}}$
$\sigma_{G_0},\ \sigma \leftarrow \mathsf{XOF}(\delta,\ \ell_{\text{pub\_seed}},\ \ell_{\text{sec\_seed}})$
$G_0 \leftarrow \mathsf{ExpandSystMat}(\sigma_{G_0})$
for $i = 1$ **to** $s-1$ **do**
    $\sigma_a,\ \sigma_{T_i},\ \sigma \leftarrow \mathsf{XOF}(\sigma,\ \ell_{\text{sec\_seed}},\ \ell_{\text{sec\_seed}},\ \ell_{\text{sec\_seed}})$
    $T_i \leftarrow \mathsf{ExpandInvMat}(\sigma_{T_i},\ k)$
    $a_{m-1,m-1} \leftarrow \mathsf{ExpandFqs}(\sigma_a,\ 1)$
    $G_0' \leftarrow T_i G_0$
    $(A_i,\ B_i^{-1}) \leftarrow \mathsf{Solve}(G_0',\ a_{m-1,m-1})$ `// Fail ⇒ retry from line 5`
    $G_i \leftarrow \mathsf{SF}(\pi_{A_i,B_i}(G_0))$ `// ⊥⇒ retry from line 5`
pk $\leftarrow (\sigma_{G_0} | \mathsf{CompressG}(G_1) | \cdots | \mathsf{CompressG}(G_{s-1}))$
sk $\leftarrow (\delta | \sigma_{G_0} | \mathsf{Compress}(A_1^{-1}) | \cdots | \mathsf{Compress}(A_{s-1}^{-1}) | \mathsf{Compress}(B_1^{-1}) | \cdots | \mathsf{Compress}(B_{s-1}^{-1}))$
return pk, sk

3 Abstract Model

We now define a unified abstraction that captures the common structure of the GGM-tree mechanism across all schemes discussed in Section 2.

Definition 2 (GZKST). *A* Generic ZK Seed Tree *(GZKST) is a tuple* $\Pi = (\mathsf{Setup}, \mathsf{Expand}, \mathsf{Chal}, \mathsf{Decide}, \mathsf{Publish}, \mathsf{Recon})$ *parametrised by:*

- λ*: the security parameter;*
- $t \in \mathbb{N}$*: the number of parallel ZK repetitions;*
- $\mathsf{G} : \{0,1\}^\lambda \to \{0,1\}^{2\lambda}$*: a secure pseudorandom generator;*
- $\mathcal{S}$*: the* secret type *(e.g., permutations for LESS, matrix pairs for MEDS, party shares for MPCitH).*

The six algorithms are:

1. $\mathsf{Setup}(1^\lambda) \to (\mathsf{seed}_r, \mathsf{salt})$*: samples a master seed* $\mathsf{seed}_r \xleftarrow{\$} \{0,1\}^\lambda$ *and a public salt.*
2. $\mathsf{Expand}(\mathsf{seed}_r, \mathsf{salt}) \to (\mathcal{T}, (\widetilde{s}_0, \ldots, \widetilde{s}_{t-1}))$*: builds a binary tree* $\mathcal{T}$ *by top-down* G*-expansion from* seed_r*. Each leaf* $i \in \{0, \ldots, t-1\}$ *holds a seed* $\widetilde{s}_i$*. Tree topology may be complete or incomplete (scheme-dependent).*
3. $\mathsf{Chal}(\mathsf{cmt}, \mathsf{msg}) \to \mathbf{c} = (c_0, \ldots, c_{t-1})$*: derives a Fiat–Shamir challenge from the commitment and message.* $c_i = 0$ *indicates that round* i *requires the ephemeral seed;* $c_i \neq 0$ *indicates a secret-dependent response.*
4. $\mathsf{Decide}(\mathbf{c}) \to (\mathcal{H}, \mathcal{R})$*: partitions* $\{0, \ldots, t-1\}$ *into the hidden index set* $\mathcal{H} = \{i : c_i \neq 0\}$ *and the revealed index set* $\mathcal{R} = \{i : c_i = 0\}$*.*
5. $\mathsf{Publish}(\mathcal{T}, \mathcal{H}) \to \mathsf{path}$*: constructs a flag structure* $\mathcal{F} : V(\mathcal{T}) \to \{0,1\}$ *and returns the minimal set of nodes* path *from which every* $\widetilde{s}_i$ *with* $i \in \mathcal{R}$ *is reconstructible, while no* $\widetilde{s}_i$ *with* $i \in \mathcal{H}$ *is derivable.*

Algorithm 2: MEDS Signing (simplified)

Input: Secret key sk, message m
Output: Signed message m_s

Parse sk to recover σ_{G_0}, and A_i^{-1}, B_i^{-1} for $i = 1, \ldots, s-1$
$G_0 \leftarrow \mathsf{ExpandSystMat}(\sigma_{G_0})$
$\delta \xleftarrow{\$} \mathcal{B}^{\ell_{\text{sec\_seed}}}$
$\rho,\ \alpha \leftarrow \mathsf{XOF}(\delta,\ \ell_{\text{tree\_seed}},\ \ell_{\text{salt}})$
$(\sigma_0, \ldots, \sigma_{t-1}) \leftarrow \mathsf{SeedTree}_t(\rho,\ \alpha)$
for $i = 0$ **to** $t-1$ **do**
    $\sigma_i' \leftarrow (\alpha | \sigma_i | \mathsf{ToBytes}(2^{1+\lceil \log_2 t \rceil} + i,\ 4))$
    $\sigma_{\tilde{A}_i},\ \sigma_{\tilde{B}_i} \leftarrow \mathsf{XOF}(\sigma_i',\ \ell_{\text{pub\_seed}},\ \ell_{\text{pub\_seed}})$
    $\tilde{A}_i \leftarrow \mathsf{ExpandInvMat}(\sigma_{\tilde{A}_i},\ m);\quad \tilde{B}_i \leftarrow \mathsf{ExpandInvMat}(\sigma_{\tilde{B}_i},\ n)$
    $\tilde{G}_i \leftarrow \mathsf{SF}(\pi_{\tilde{A}_i, \tilde{B}_i}(G_0))$ `// ⊥⇒ resample` σ_i'
$d \leftarrow H(\mathsf{Compress}(\tilde{G}_0[\,;k, mn-1]) | \cdots | \mathsf{Compress}(\tilde{G}_{t-1}[\,;k, mn-1]) | m)$
$(h_0, \ldots, h_{t-1}) \leftarrow \mathsf{ParseHash}_{s,t,w}(d)$
for $i = 0$ **to** $t-1$ ***with*** $h_i > 0$ **do**
    $\mu_i \leftarrow \tilde{A}_i \cdot A_{h_i}^{-1};\quad \nu_i \leftarrow B_{h_i}^{-1} \cdot \tilde{B}_i$
    Append $(\mathsf{Compress}(\mu_i) | \mathsf{Compress}(\nu_i))$ to response vector v
$p \leftarrow \mathsf{SeedTreeToPath}_t(h_0, \ldots, h_{t-1},\ \rho,\ \alpha)$
return $m_s \leftarrow (v | p | d | \alpha | m)$

6. $\mathsf{Recon}(\mathsf{path}, \mathbf{c}, \mathsf{salt}) \to \{\widetilde{s}_i : i \in \mathcal{R}\}$: *verifier-side reconstruction of revealed leaf seeds from* path.

3.1 The Flag Structure

The flag structure $\mathcal{F}$ produced internally by Publish is the key intermediate object. It is a labelling $\mathcal{F} : V(\mathcal{T}) \to \{0, 1\}$ where:

- $\mathcal{F}(v) = 0$: node v is *publishable*—either directly included in path or derivable from a published ancestor;
- $\mathcal{F}(v) = 1$: node v is *hidden*—it must not appear in path and must not be derivable from published nodes.

The flag structure satisfies two invariants:

Invariant 1 (Correctness). For every $i \in \mathcal{R}$ there exists a node v on the path from the root to leaf $\widetilde{s}_i$ such that $\mathcal{F}(v) = 0$ and $\mathcal{F}(\mathsf{parent}(v)) = 1$.

Invariant 2 (ZK hiding). For every $i \in \mathcal{H}$, no ancestor of leaf $\widetilde{s}_i$ satisfies $\mathcal{F}(v) = 0$.

The ZK property of the signature scheme is contingent on Invariant 2 holding for *all* $i \in \mathcal{H}$. A fault that changes $\mathcal{F}(v)$ from 1 to 0 for any v that is an ancestor of some $\widetilde{s}_i$ with $i \in \mathcal{H}$ *breaks Invariant 2* and may allow an attacker to reconstruct $\widetilde{s}_i$.

Algorithm 3: MEDS Verification (simplified)

Input: Public key pk, signed message m_s
Output: Message m or $\perp$

$G_0 \leftarrow \mathsf{ExpandSystMat}(\mathsf{pk}[0, \ell_{\mathrm{pub\_seed}} - 1])$
for $i = 1$ **to** $s - 1$ **do**
  $G_i \leftarrow \mathsf{DecompressG}(\mathsf{pk}[\cdots])$
Parse m_s to obtain: response pairs $((\mu_i, \nu_i))_{h_i > 0}$, path p, digest d, salt α, message m
$(h_0, \ldots, h_{t-1}) \leftarrow \mathsf{ParseHash}_{s,t,w}(d)$
$(\sigma_0, \ldots, \sigma_{t-1}) \leftarrow \mathsf{PathToSeedTree}_t(h_0, \ldots, h_{t-1},\ p,\ \alpha)$
for $i = 0$ **to** $t - 1$ **do**
  if $h_i > 0$
    $(\mu_i,\ \nu_i) \leftarrow$ next response pair from m_s
    if $\mu_i \notin \mathrm{GL}_m(q)$ ***or*** $\nu_i \notin \mathrm{GL}_n(q)$ **return** $\perp$
    $\hat{G}_i \leftarrow \mathsf{SF}(\pi_{\mu_i,\nu_i}(G_{h_i}))$
    if $\hat{G}_i = \perp$ **return** $\perp$
  else
    $\sigma'_i \leftarrow (\alpha | \sigma_i | \mathsf{ToBytes}(2^{1+\lceil \log_2 t \rceil} + i,\ 4))$
    $\hat{A}_i \leftarrow \mathsf{ExpandInvMat}(\mathsf{XOF}(\sigma'_i)_1,\ m);\quad \hat{B}_i \leftarrow \mathsf{ExpandInvMat}(\mathsf{XOF}(\sigma'_i)_2,\ n)$
    $\hat{G}_i \leftarrow \mathsf{SF}(\pi_{\hat{A}_i,\hat{B}_i}(G_0))$
$d' \leftarrow H(\mathsf{Compress}(\hat{G}_0[\,;k, mn-1]) | \cdots | \mathsf{Compress}(\hat{G}_{t-1}[\,;k, mn-1]) | m)$
if $d = d'$ **return** m
return $\perp$

3.2 Mapping Schemes to the GZKST Model

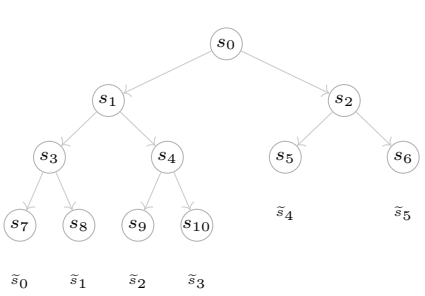
We now establish that each of the ZK-based post-quantum signature schemes introduced in Section 2 is a valid GZKST instance in the sense of Definition 2. For each scheme, we exhibit the concrete instantiation of the six algorithms and verify that Invariants 1 and 2 hold by construction.

LESS-V2

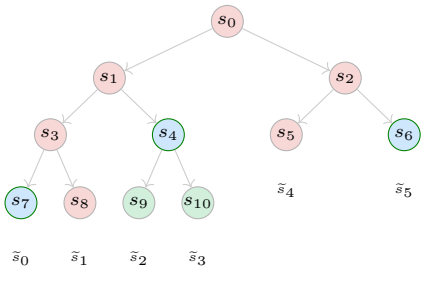
Proposition 1 (LESS-v2 is a GZKST instance). *LESS-v2 instantiates Definition 2 with secret type $\mathcal{S} = \{\pi_1, \ldots, \pi_{s-1}\}$ (the $s-1$ secret monomial permutations), t parallel repetitions, and an* incomplete GGM *tree of height $\lceil \log_2 t \rceil$. Invariants 1 and 2 hold by construction of the three-phase flag tree; see Appendix A.1 for the full proof.*

Remark 1 (Invariants under incomplete trees). When the tree is incomplete, some leaves appear at depth $\lceil \log_2 t \rceil - 1$. The index-tracking arrays npl, ll, off, and lsi ensure that LabelLeaves and ComputeSeeds visit the correct nodes regardless of level. Both invariants therefore hold for any tree shape produced by the LESS-v2 expansion (see Appendix A.1).

- $\mathsf{Setup}(1^\lambda) \to (\mathsf{seed}_r, \mathsf{salt})$: The root seed is derived from the secret key seed; the salt is sampled uniformly at random.
- $\mathsf{Expand}(\mathsf{seed}_r, \mathsf{salt}) \to (\mathcal{T}, (\widetilde{s}_0, \ldots, \widetilde{s}_{t-1}))$: LESS-v2 uses an incomplete GGM tree with $\lceil \log_2 t \rceil$ levels. Leaves are distributed across the bottom two levels; index offsets are tracked via npl, ll, off, and lsi (Figure 2a).



(a) Expanded GGM seed tree.



(b)

After Chal, Decide, Publish; $\mathbf{b}=\{0,1,0,0,1,0\}$, $\mathsf{path}=\{s_4, s_6, s_7\}$.

Fig. 2: LESS-v2 GGM seed tree ($t = 6$): structure (a) and flag state (b). In (b): pink nodes ($\mathcal{F}$=1, hidden subtree); green nodes ($\mathcal{F}$=0, fully revealed subtree); blue-bordered nodes ($\mathcal{F}$=0, in path).

- $\mathsf{Chal}(\mathsf{cmt}, \mathsf{msg}) \to \mathbf{c}$: The t canonical-form matrices together with the salt and message are hashed; SampleChallenge produces $\mathbf{b} \in \{0, \ldots, s-1\}^t$ of weight w. Setting $c_i = b^{(i)}$, the condition $c_i = 0$ (resp. $c_i \neq 0$) signals a revealed (resp. hidden) round.
- $\mathsf{Decide}(\mathbf{c}) \to (\mathcal{H}, \mathcal{R})$: $\mathcal{H} = \mathrm{supp}(\mathbf{b})$, $\mathcal{R} = \{0, \ldots, t-1\} \setminus \mathcal{H}$.
- $\mathsf{Publish}(\mathcal{T}, \mathcal{H}) \to \mathsf{path}$: LESS-v2 builds an auxiliary flag tree of the same shape via two phases: (i) *leaf labelling* (LabelLeaves): $\mathcal{F}(\mathrm{leaf}_i) = \mathbf{1}[i \in \mathcal{H}]$; (ii) *upward propagation* (ComputeSeeds): $\mathcal{F}(v) = \mathcal{F}(\ell) \vee \mathcal{F}(r)$ for each internal node v with children ℓ and r. The two-phase description is an implementation detail; the resulting flag assignment is exactly the OR-propagation of equations (1)–(2) (the root's flag is determined solely by this propagation). The path $\mathsf{path} = \{v : \mathcal{F}(v) = 0,\ \mathcal{F}(\mathrm{parent}(v)) = 1\}$ (with the convention that $\mathcal{F}(\mathrm{parent}(\mathrm{root})) = 1$) is the minimal publishable boundary (Figure 2b).
- $\mathsf{Recon}(\mathsf{path}, \mathbf{c}, \mathsf{salt}) \to \{\tilde{s}_i : i \in \mathcal{R}\}$: PathToSeedTree places received nodes on an empty tree of the same shape and re-expands downward via G until all t leaves are filled. Positions $i \in \mathcal{H}$ remain empty; positions $i \in \mathcal{R}$ yield the recovered seeds from which monomial matrices and commitments are re-derived.

MEDS

Proposition 2 (MEDS is a GZKST instance). *MEDS instantiates Definition 2 with secret type $\mathcal{S} = \{(A_j^{-1}, B_j^{-1})\}_{j=1}^{s-1}$, t parallel repetitions, and a GGM tree of height $\lceil \log_2 t \rceil$ (complete when t is a power of two, incomplete otherwise; only the first t of the $2^{\lceil \log_2 t \rceil}$ leaves are used). Invariants 1 and 2 hold by the SeedTreeToPath construction regardless of whether the tree is complete or incomplete; see Appendix A.2.*

- $\mathsf{Setup}(1^\lambda) \to (\mathsf{seed}_r, \mathsf{salt})$: MEDS samples a fresh random scalar and derives the tree seed and salt via a single XOF call.

- $\mathsf{Expand}(\mathsf{seed}_r, \mathsf{salt}) \to (\mathcal{T}, (\widetilde{s}_0, \ldots, \widetilde{s}_{t-1}))$: A binary tree of height $\lceil \log_2 t \rceil$ is built by hashing each parent with the salt and node index; the first t leaves are returned. When t is not a power of two, the rightmost $2^{\lceil \log_2 t \rceil} - t$ potential leaves are unused, making the tree incomplete—the same structural situation as LESS-v2.
- $\mathsf{Chal}(\mathsf{cmt}, \mathsf{msg}) \to \mathbf{c}$: Each leaf is expanded into sub-seeds via XOF, ephemeral matrices are sampled, and the commitment is the hash of all compressed generator matrices together with m. Parsing the digest via $\mathsf{ParseHash}_{s,t,w}$ yields $\mathbf{h} \in \{0, \ldots, s-1\}^t$ of weight w; setting $c_i = h_i$, $c_i = 0$ (resp. $c_i \neq 0$) signals a revealed (resp. hidden) round.
- $\mathsf{Decide}(\mathbf{c}) \to (\mathcal{H}, \mathcal{R})$: $\mathcal{H} = \{i : c_i \neq 0\}$, $\mathcal{R} = \{i : c_i = 0\}$.
- $\mathsf{Publish}(\mathcal{T}, \mathcal{H}) \to \mathsf{path}$: $\mathsf{SeedTreeToPath}_t$ removes the label of each leaf $i \in \mathcal{H}$ and propagates each deletion upward, erasing every ancestor that has at least one hidden leaf in its subtree (i.e., the same bottom-up $\mathcal{F}(v) = \mathcal{F}(\ell) \vee \mathcal{F}(r)$ rule as LESS-v2). Surviving boundary nodes—those with $\mathcal{F}(v) = 0$ and $\mathcal{F}(\mathrm{parent}(v)) = 1$—are serialised in depth-first order.
- $\mathsf{Recon}(\mathsf{path}, \mathbf{c}, \mathsf{salt}) \to \{\widetilde{s}_i : i \in \mathcal{R}\}$: $\mathsf{PathToSeedTree}_t$ places path nodes on an empty tree and re-expands downward via G; positions $i \in \mathcal{H}$ remain empty.

CROSS

Proposition 3 (CROSS is a GZKST instance). *CROSS instantiates Definition 2 with secret type $\mathcal{S} = \{\mathbf{e}\}$ (the R-SDP solution), t parallel repetitions, and a complete GGM tree. The seed-publishing component satisfies Invariants 1 and 2 by the same argument as MEDS (Proposition 2). The additional Merkle proof over $\{\mathsf{cmt}_0[i]\}_{i \in \mathcal{H}}$ is a CROSS-specific commitment-authenticity mechanism orthogonal to the GZKST seed-hiding invariants.*

- $\mathsf{Setup}(1^\lambda) \to (\mathsf{seed}_r, \mathsf{salt})$: Root seed of size λ bits and salt of size 2λ bits, both drawn uniformly at random.
- $\mathsf{Expand}(\mathsf{seed}_r, \mathsf{salt}) \to (\mathcal{T}, (\widetilde{s}_0, \ldots, \widetilde{s}_{t-1}))$: A complete binary tree (CROSS-v1 identical to LESS-v1; CROSS-v2 uses the SeedTreePath mechanism of MEDS) where each parent seed is concatenated with the salt and node index and processed by SHAKE to yield two λ-bit child seeds.
- $\mathsf{Chal}(\mathsf{cmt}, \mathsf{msg}) \to \mathbf{c}$: CROSS applies Fiat–Shamir in two rounds. The t commitment pairs $(\mathsf{cmt}_0[i], \mathsf{cmt}_1[i])$, the salt, and the message are hashed to produce $\mathsf{chall}_1 \in (\mathbb{F}_p^*)^t$, from which first-response digests $y_i = \mathrm{Hash}(\mathbf{u}'_i + \mathsf{chall}_1[i] \cdot \mathbf{e}'_i)$ are derived. A second hash yields $\mathsf{chall}_2 \in \mathcal{B}(t, w)$; setting $c_i = \mathsf{chall}_2[i] \in \{0, 1\}$, $c_i = 0$ (resp. 1) signals a revealed (resp. hidden) round.
- $\mathsf{Decide}(\mathbf{c}) \to (\mathcal{H}, \mathcal{R})$: $\mathcal{H} = \{i : c_i = 1\}$, $\mathcal{R} = \{i : c_i = 0\}$.
- $\mathsf{Publish}(\mathcal{T}, \mathcal{H}) \to \mathsf{path}$: The seed path is computed as in MEDS. Additionally, a Merkle proof over $\{\mathsf{cmt}_0[i]\}_{i \in \mathcal{H}}$ authenticates the w commitment values that the verifier cannot recompute; this is orthogonal to the GZKST seed-hiding invariants (see Proposition 3).

– $\mathsf{Recon}(\mathsf{path}, \mathbf{c}, \mathsf{salt}) \to \{\widetilde{s}_i : i \in \mathcal{R}\}$: Path nodes are placed on an empty tree and re-expanded downward. Positions $i \in \mathcal{H}$ remain empty; positions $i \in \mathcal{R}$ yield seeds from which $(\mathbf{e}'_i, \mathbf{u}'_i)$ are re-derived and $\mathsf{cmt}_1[i]$ is recomputed for verification.

Definition 3 (Faulted Signing Oracle). *Fix a node $v^* \in V(\mathcal{T})$. The* faulted signing oracle $\mathcal{O}^{v^*}(\mathsf{sk}, \mathsf{pk})$ *takes a message* msg *and executes the signing algorithm with a single modification: during* $\mathsf{Publish}(\mathcal{T}, \mathcal{H})$*, the flag value is forced to*

$$\mathcal{F}(v^*) \leftarrow 0,$$

regardless of whether $\mathcal{F}(v^*)$ equals 1 in the honest execution. Upward propagation is applied as normal after this assignment. The oracle returns the resulting (possibly faulted) signature sig^*.

3.3 Oracle Abstraction

We previously presented the abstract view of the GGM-tree and its instantiations in LESS, MEDS, and CROSS. We now formalize the attacker's interface in an abstract manner, extending the GZKSTtuple with a fault model and a secret-extraction procedure.

Faulted signing oracle. Let $\Pi = (\mathsf{Setup}, \mathsf{Expand}, \mathsf{Chal}, \mathsf{Decide}, \mathsf{Publish}, \mathsf{Recon})$ be a GZKST instance with secret type $\mathcal{S}$, and let $(\mathsf{sk}, \mathsf{pk})$ be a fixed key pair. We model the adversary's access to the victim's signing device as a *faulted oracle*:

Definition 4 (Faulted Signing Oracle). *Fix a node $v^* \in V(\mathcal{T})$. The* faulted signing oracle $\mathcal{O}^{v^*}(\mathsf{sk}, \mathsf{pk})$ *takes a message* msg *and executes the signing algorithm with a single modification: during* $\mathsf{Publish}(\mathcal{T}, \mathcal{H})$*, the flag value is forced to*

$$\mathcal{F}(v^*) \leftarrow 0,$$

regardless of whether $\mathcal{F}(v^*)$ equals 1 in the honest execution. Upward propagation is applied as normal after this assignment. The oracle returns the resulting (possibly faulted) signature sig^*.

The fault is *effective* if the original value $\mathcal{F}(v^*)$ was 1 and v^* is an ancestor of at least one leaf $\widetilde{s}_i$ with $i \in \mathcal{H}$: in this case, the published path path now contains a node from which $\widetilde{s}_i$ is derivable, directly violating Invariant 2. The fault is *ineffective* otherwise (the node was already publishable, or all hidden leaves lie in the sibling subtree).

Remark 2. The fault model of Definition 4 is realised by several concrete physical mechanisms: an instruction-skip fault that bypasses the store $\mathcal{F}(v) \leftarrow \mathcal{F}(\ell) \vee \mathcal{F}(r)$, a stuck-at-zero perturbation forcing $\mathcal{F}(v^*) = 0$, a bit-flip $(1 \to 0)$ via Rowhammer, or a clock-glitch that skips the validity check on a specific tree node. All of these are documented in the literature [10,11].

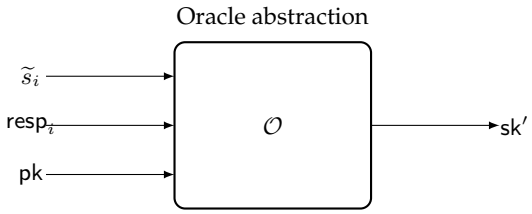


Fig. 3: The Extract oracle: given dual revelation $(\widetilde{s}_i, \mathsf{resp}_i)$ and public key pk, it outputs partial secret sk'.

Remark 3. We assume that our faulted oracle has an algorithm `SolverS` that efficiently solves secrets of type $\mathcal{S}$. For example, when instantiating our oracle against LESS, we use Lemma 1 from [10] to solve the secret type.

Dual revelation. The invariant that the Publish phase must enforce is that, for every hidden round $i \in \mathcal{H}$, the signer simultaneously publishes:

(i) the ZK *response* resp_i, which encodes the relationship between $\widetilde{s}_i$ and the long-term secret sk; and
(ii) a path path from which $\widetilde{s}_i$ is *not* derivable.

An effective fault breaks condition (ii): the attacker now holds both $\widetilde{s}_i$ (reachable from the faulted path) and resp_i (always present in the signature). This *dual revelation* is the primitive from which all secret-extraction procedures in this work are built.

Secret extraction. We extend the GZKST tuple with a seventh, scheme-specific procedure:

Definition 5 (Extract). $\mathsf{Extract}(\widetilde{s}_i, \mathsf{resp}_i, \mathsf{pk}) \to \mathsf{sk}'$*: given the leaked seed* $\widetilde{s}_i$ *and the ZK response* resp_i *for the same round* $i \in \mathcal{H}$*, recover the (partial) secret* $\mathsf{sk}' \subseteq \mathcal{S}$*.*

For the schemes studied in this work, Extract takes the following concrete forms:

– **MEDS**. For a hidden round i with $h_i = j$, the signer publishes $(\mu_i, \nu_i) = (A_j \widetilde{A}_i^{-1},\ B_j^{-1} \widetilde{B}_i)$ (consistent with the background description in §2.2). Expanding $\widetilde{s}_i$ yields $(\widetilde{A}_i, \widetilde{B}_i)$, so

$$A_j \;=\; \mu_i \cdot \widetilde{A}_i, \qquad B_j^{-1} \;=\; \nu_i \cdot \widetilde{B}_i^{-1}.$$

  Both operations are direct matrix multiplications (and one inversion) over $\mathbb{F}_q$; no column-reconstruction step is required.
– **LESS-v2**. The response is a coset representative $\mathsf{CosetRep}(\pi_{b[i]} \circ \tilde{\pi}_i^*)$. Expanding $\widetilde{s}_i$ gives the partial monomial matrix $\tilde{\pi}_i^*$, and the secret permutation $\pi_{b[i]}$ is recovered via the column-extraction argument of Lemma 1 in [11].
– **CROSS**. The response encodes $\mathbf{v}_i = \mathbf{e} \star (\mathbf{e}'_i)^{-1}$. Expanding $\widetilde{s}_i$ gives $\mathbf{e}'_i$, so $\mathbf{e} = \mathbf{v}_i \star \mathbf{e}'_i$ directly.

Algorithm 4: GZKST key recovery from faulted oracle

Input : Faulted oracle $\mathcal{O}^{v^*}$, public key pk
Output : Recovered secret key $\widehat{\mathsf{sk}}$

$\mathsf{Collected} \leftarrow \emptyset$
while $|\mathsf{Collected}| < |\mathcal{S}|$ **do**
  $\mathsf{sig}^* \leftarrow \mathcal{O}^{v^*}(\mathsf{msg})$ for an arbitrary msg
  if sig^* *is* effective
    for *each* $i \in \mathsf{leaked}(\mathsf{sig}^*)$ **do**
      $\mathsf{sk}' \leftarrow \mathsf{Extract}(\widetilde{s}_i, \mathsf{resp}_i, \mathsf{pk})$
      $\mathsf{Collected} \leftarrow \mathsf{Collected} \cup \{(j, \mathsf{sk}')\}$ where $j = c_i$
return $\widehat{\mathsf{sk}} \leftarrow \mathsf{Collect}$

Full key recovery. When $|\mathcal{S}| = s - 1 > 1$ (as in MEDS with $s \geq 3$, and LESS with $s \geq 3$), a single effective fault recovers only the secret indexed by h_i. Full recovery requires observing each index $j \in \{1, \ldots, s-1\}$ at least once.

Proposition 4 (Correctness of Algorithm 4). *Let Π be a* GZKST *instance, suppose $v^* = \mathrm{root}$ (so the fault exposes all w hidden leaves, making $L = w$ deterministic), and let $p_{\min} = 1 - \left(\frac{s-2}{s-1}\right)^w$. Algorithm 4 terminates and outputs $\widehat{\mathsf{sk}} = \mathsf{sk}$ with probability at least $1 - \delta$ after at most $\left\lceil \frac{s-1}{p_{\min}} \cdot \ln\frac{1}{\delta} \right\rceil$ effective oracle queries; see Appendix A.4 for the full proof.*

Remark 4 (General fault location). For an effective fault at an arbitrary node v^* with ℓ leaf descendants, the number $L = |\mathcal{H} \cap \mathrm{leaves}(v^*)|$ of hidden leaves in the subtree is a hypergeometric random variable whose realisation may be smaller than its mean $\bar{w} = w\ell/t$, making the per-query success probability $p_r(L) = 1 - \big((s-1-r)/(s-1)\big)^L$ potentially smaller than the root-fault $p_{\min}$. A valid bound for any effective fault is obtained by taking the worst case $L = 1$, which gives $p_r \geq r/(s-1)$ for all $r \geq 1$ and therefore $\mathbb{E}[Q] \leq (s-1)\, H_{s-1}$, where $H_{s-1} = \sum_{r=1}^{s-1} 1/r$ is the $(s-1)$-th harmonic number. For $s \leq 6$ (all schemes in Table 1), this gives $\mathbb{E}[Q] \leq 5H_5 \approx 11.4$, still a small constant.

Detecting whether a received signature is effective is feasible for the adversary: given the fault location v^* and the challenge $\mathbf{c}$ (recoverable from the digest d in the signature), the adversary recomputes the honest flag tree, checks whether $\mathcal{F}(v^*) = 1$, and verifies that at least one hidden leaf falls in the subtree of v^*. This detection is detailed for LESS in [10] and applies unchanged to MEDS and CROSS.

Expected query count. Let ℓ denote the number of leaves in the subtree rooted at v^*.

Proposition 5. *Let $v^* = \mathrm{root}$, so $L = w$ exactly. Define $p_{\min} = 1 - \left(\frac{s-2}{s-1}\right)^w$. The per-query success probability satisfies $p_r \geq p_{\min}$ for all $r \geq 1$, and the expected total number of effective queries satisfies*

$$\mathbb{E}[Q] \;\leq\; \frac{s-1}{p_{\min}}.$$

See Appendix A.5 for the derivation.

Remark 5 (Role of E_{dist} and Jensen's inequality). The quantity $E_{\mathrm{dist}} = (s-1)\,p_{\min}$ equals the expected number of distinct secret indices revealed per effective query (when $L = w$ is deterministic at the root). When $v^* \neq \mathrm{root}$ and L is random with mean $\bar{w} = w\ell/t$, Jensen's inequality (the function α^x is convex in x) gives $\mathbb{E}[\#\text{distinct}] \leq E_{\mathrm{dist}}$: E_{dist} is an *upper* bound on expected per-query gain, which implies a *lower* bound on $\mathbb{E}[Q]$, not an upper bound. The upper bound $\mathbb{E}[Q] \leq (s-1)/p_{\min}$ stated in Proposition 5 follows from the direct inequality $p_r \geq p_{\min}$, not from the Jensen argument.

For all MEDS parameter sets (Table 1), faulting the root ($v^* = \mathrm{root}$, $\ell = t$) gives $\bar{w} = w$; for those parameters $w \gg 1$, so $p_{\min} \approx 1$ and $\mathbb{E}[Q] \lesssim s - 1$, which is at most 5 across all sets. In practice a single effective query suffices with high probability, as confirmed by the simulation in §5.

Remark 6 (Concrete query counts for MEDS). Table 2 reports $p_{\min}$ and $\mathbb{E}[Q]$ for every MEDS parameter set when $v^* = \mathrm{root}$ (so $L = w$ exactly). The values use the exact formula $p_{\min} = 1 - \big((s-2)/(s-1)\big)^w$; for all parameter sets $p_{\min} > 0.99$, giving $\mathbb{E}[Q] \leq s - 1 \leq 5$.

Table 2: Expected effective queries $\mathbb{E}[Q]$ for MEDS (fault at root).

Parameter set	s	w	$s-1$	$p_{\min}$	$\mathbb{E}[Q] \leq$
MEDS-9923	4	14	3	0.9966	3.01
MEDS-13220	5	20	4	0.9968	4.01
MEDS-41711	4	26	3	1.0000	3.00
MEDS-69497	5	36	4	1.0000	4.00
MEDS-134180	5	52	4	1.0000	4.00
MEDS-167717	6	66	5	1.0000	5.00

From key recovery to forgery. Once Algorithm 4 terminates, the adversary holds a complete signing key $\widehat{\mathsf{sk}}$ and can produce valid signatures on any message msg^* — including messages never submitted to any oracle. This constitutes a full break of EUF-CMA:

Corollary 1. *Let Π be a* GZKST*-based signature scheme and let the fault model be as in Definition 4 (a stuck-at-zero perturbation of a single flag bit during* Publish*). If an efficient adversary can induce this fault at a fixed node $v^* \in V(\mathcal{T})$ on the victim's signing device, then Π is* not *EUF-CMA-secure in the fault model: the adversary recovers* sk *via Algorithm 4 and forges signatures on arbitrary messages without further oracle access.*

4 Fault Model Taxonomy

4.1 Comparative study of the attack surfaces in state of the art attacks

Table 3: Fault Attack mapping of attacks on GZKST-Based Signature Schemes

	LESS	**MEDS**	**CROSS**	**RYDE/ Mirath**	**FAEST**
ZKFault (Mondal et al.)	Publish	Publish	Publish (not tested)	-	-
Fault to Forge (Mondal et al.)	Publish	-	-	-	-
Faultless Key Recovery (Huang et al.)	Decide	-	-	-	-
Fault Attack on MPCitH (Banda et al.)	-	-	-	Expand, Chal	-
A Case Study on FAEST (Jendral & Dubrova)	-	-	-	-	Expand

5 Fault Injection attacks on MEDS GGM tree procedures

This subsection details a fault injection (FI) attack conducted on the C reference implementation of the MEDS signature algorithm. We focus on exploiting the tree traversal logic used in the path construction to leak hidden seeds.

5.1 Target Analysis

The primary target of our analysis is the SeedTreePath function, specifically targeting the logic represented in the signing phase of the MEDS algorithm (instruction 16 of algorithm 2). In the reference implementation, both SeedTreePath (path generation) and SeedTree (tree reconstruction) are handled by a unified function, `stree_to_path_to_stree`, which operates in two modes defined by a `mode` parameter.

Listing 1.1: `stree_to_path_to_stree` implementation

```
void stree_to_path_to_stree(uint8_t *stree, uint8_t *h_digest, uint8_t *path,
     uint8_t *salt, int mode) {
    int h = 0, i = 0;
    unsigned int id = 0, idx = 0;
    unsigned int indices[MEDS_w] = {0};
    for (int i = 0; i < MEDS_t; i++) {
        if (h_digest[i] > 0) {
            indices[idx++] = i;
        }
    }
    while (1==1) {
        int index_leaf = 0;
        while ((indices[id] » (MEDS_seed_tree_height-h)) == i)
        {// Traverse down toward the secret leaf
            h += 1;
            i <<= 1;
            if (h > MEDS_seed_tree_height) {
                h -= 1;
                i >>= 1;
                if (id + 1 < MEDS_w) id++;
                index_leaf = 1; // Flag node as hidden
                break;
            }
        }
        if (index_leaf == 0) {
            if (mode == STREE_TO_PATH) {
                memcpy(path, &stree[MEDS_st_seed_bytes * SEED_TREE_ADDR(h, i)],
     MEDS_st_seed_bytes);
                path += MEDS_st_seed_bytes;
            } else {
                memcpy(&stree[MEDS_st_seed_bytes * SEED_TREE_ADDR(h, i)], path,
     MEDS_st_seed_bytes);
                path += MEDS_st_seed_bytes;
                t_hash(stree, salt, h, i);
            }
        }
        // Backtracking logic
        while ((i & 1) == 1) {
            h -= 1;
            i >>= 1;
        }
        i += 1;
        if ((i << (MEDS_seed_tree_height - h)) >= MEDS_t) return;
    }
}
```

The algorithm identifies secret leaves (where $h[i] > 0$) and performs a Depth-First Search (DFS). The exploration halts as soon as it reaches a node that is no longer an ancestor of the hidden leaf. This node is then appended to the signature path. If the exploration reaches a target hidden leaf, the `index_leaf` flag is set to ensure the leaf seed remains unpublished and we move on to the next hidden leaf.

Objective The adversary's goal is to manipulate the control flow via fault injection to leak unpublished seeds (internal nodes or leaves) into the public signature path, ultimately compromising the security of the signature scheme.

5.2 Experimental Setup

Our experimental platform consists of a **CW303-STM32F3** target board, featuring an ARM Cortex-M4 STM32F303RCT7 processor. Clock glitching is performed using a **ChipWhisperer-Lite**. As a proof-of-concept, the target board is flashed exclusively with the `stree_to_path_to_stree` function, using fixed values for the reference tree, salt, and challenge.

5.3 Fault Models

We propose three fault models targeting different stages of the tree traversal.

Model 1: Full Tree Recovery (Root Leakage) This attack aims to reveal the root seed, which effectively compromises the entire tree. By injecting a fault that causes the skip of the first iteration of `while` loop at line 13 of listing 5.1, the DFS halts immediately at the root ($h = 0, i = 0$). The root is incorrectly identified as a node ready for publication.

Detection and recovery. The signature path contains only a single non-zero element. Verification is performed by expanding this single seed into a full tree and checking if the resulting signature is accepted.

Model 2: Hidden Leaf Recovery This model targets the specific instruction `index_leaf = 1;` at line 20 of listing 5.1. By skipping this assignment, a hidden leaf is no longer marked as private. The execution continues normally, but the secret leaf seed is copied into the `path` buffer.

Detection and recovery. The path length will be $w+k$ (where $k > 0$ is the number of iterations of the target instruction that were skipped). Detection involves testing subsets of the path to find a valid signature, which may be computationally expensive ($\binom{w+k}{w}$ verifications).

Model 3: Subtree Recovery (Index Corruption) By glitching the loop `if (id + 1 < MEDS_w)` at line 19 of listing 5.1, the algorithm fails to update the target index `id`. The traversal logic becomes desynchronized, resulting in the algorithm treating subsequent branches as public. All leaves to the right of the faulted `id` become computable.

Detection and recovery. This fault is hard to detect without a reference signature. However, one can simulate tree reconstructions for all possible faulted `id` values to verify which leads to a valid signature. (up to w verifications)

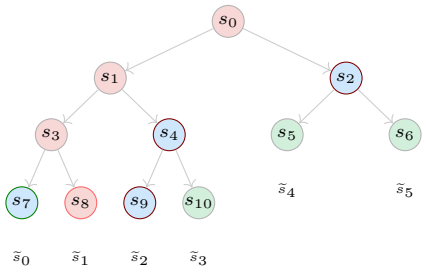


Fig. 4: Effect of fault model 3 on the example at 2, if the glitch occured while `id==1`. The returned path is $\{s_7, s_9, s_4, s_2\}$ and all leaves are computable except x_1.

5.4 Results and Observations

We successfully executed the fault models on the Chipwhisperer board. This section detailed the compiler and glitcher configurations that were required to obtain the desired faults. All three fault models were successfully reproduced.

Full tree recovery

Compilation. This attack is carried out on a code compiled with the default `-Os` optimization.

Trigger position. We have found the most efficient way to make the while condition fail from the first iteration is to fault the initialization of the involved variables (corresponding to lines 2 and 3 of 5.1). The compiled result is in 5.4.

```
0800062c <stree_to_path_to_stree_glitch>:
...
 8000704:   f001 fa18   bl  8001b38 <trigger_high> // trigger_high();
 8000708:   f04f 0800   mov.w   r8, #0             // unsigned int id = 0;
 800070c:   4645        mov r5, r8                 // int i = 0;
 800070e:   4647        mov r7, r8                 // int h = 0;
...
```

Parameters. We fault for 4 clock cycles. A glitch is successful if the returned path contains the root node. Figure 5 shows that the attack was successful for :

- An offset neighboring -13.5 with a positive offset
- A width neighboring 13.5 with a negative offset
- A sum of a width and offset of approximatively 13.5

Todo. Interpretation.

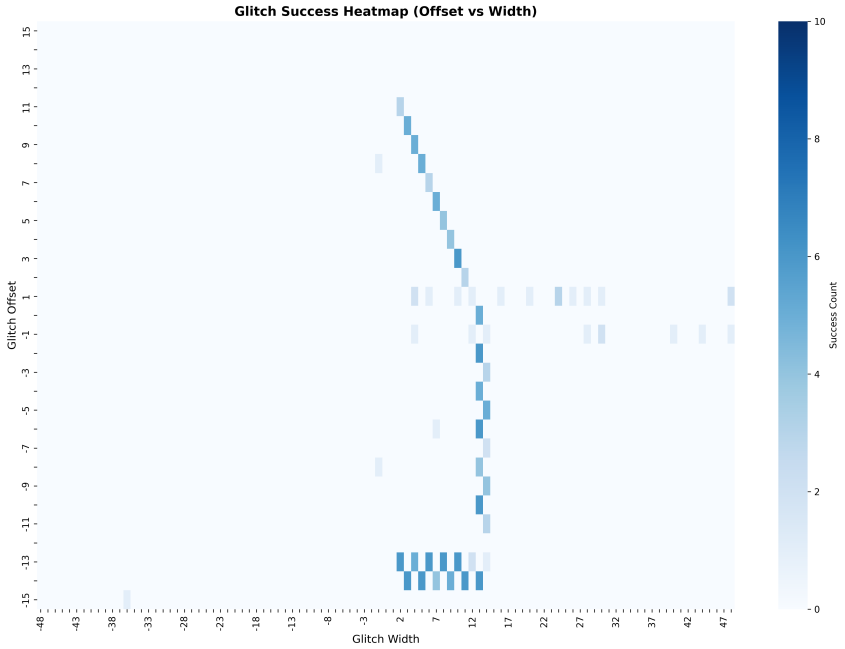


Fig. 5: Glitch success frequency by glitch width and offset for fault model 1, with 10 tries per parameter pair.

Subtree recovery

Compilation. This is carried out on a code compiled with the default `-Os` optimization.

Trigger position. We set the trigger to frame lines 19 of 5.1. The compiled result is in 5.4.

```
080006d2 <stree_to_path_to_stree_glitch>:
...
 8000726:   f001 fa09   bl  8001b3c <trigger_high> // trigger_high();
 800072a:   f108 0401   add.w   r4, r8, #1
 800072e:   2c05        cmp r4, #5                 //if (id+1 < MEDS_w)
 8000730:   bf88        it  hi
 8000732:   4644        movhi   r4, r8             // id++ ;
 8000734:   f001 fa09   bl  8001b4a <trigger_low>  // trigger_low();
...
```

Parameters. The evaluation of the condition and the incrementation of `id` require 3 clock cycles. We therefore set `repeat` to 3. We consider the attack successful if we obtain the faulted path pattern described by figure 4.

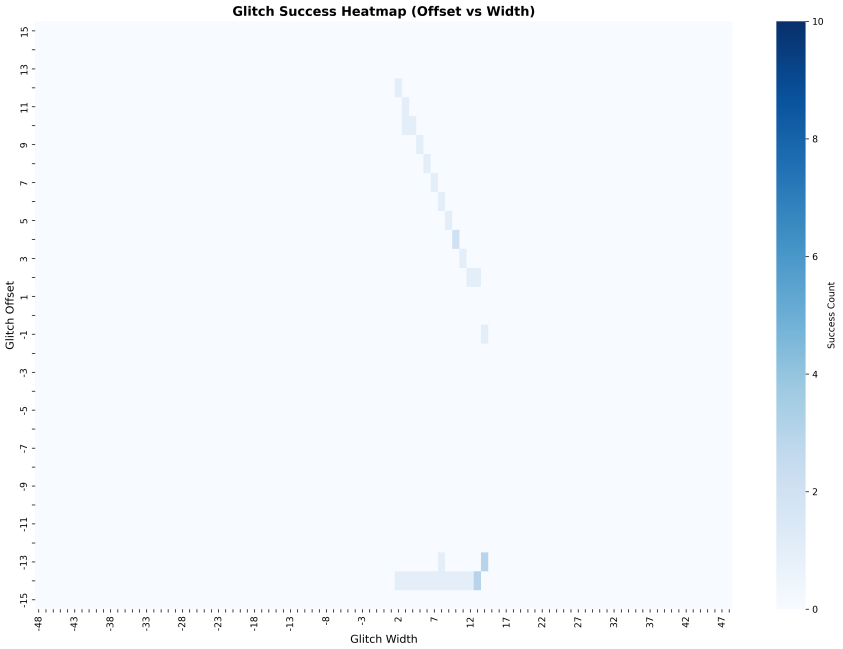


Fig. 6: Glitch success frequency by glitch width and offset for fault model 2, with 10 tries per parameter pair.

Single hidden leaf recovery

Compilation. Higher compiler optimization levels made it difficult to precisely target the flags. Specifically, the `-O1`, `-O2`, `-O3`, and `-Os` optimization flags resulted in compiled assembly code that omitted the explicit assignment instruction entirely, relying instead on complex control-flow manipulations to achieve the same logical outcome. To resolve this issue, we enforced the `-O0` optimization level strictly on the targeted function. Applying `-O0` to the entire project was unfeasible, as it increased the binary size beyond the storage capacity of the target board.

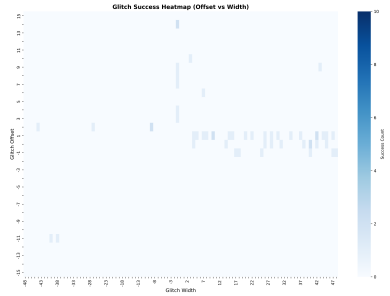
Trigger position. For the unoptimized version, we position the glitch high trigger right before the `index_leaf` assignment and the low trigger right after. This results in the compiled code in listing 5.4.

```
0800062c <stree_to_path_to_stree_glitch>:
...
 80006be:   f001 fd3f   bl  8002140 <trigger_high> // trigger_high();
 80006c2:   2301        movs    r3, #1             // index_leaf = 1;
 80006c4:   62bb        str r3, [r7, #40]   @ 0x28
 80006c6:   f001 fd43   bl  8002150 <trigger_low>  // trigger_low();
...
```

Parameters. The disassembled code shows that the assignement is done in two instructions, first The CPU loads the immediate integer value 1 directly into the register `r3`, then it takes that value 1 from `r3` and writes it out to memory. According to ARM documentation, each of these instruction is 1 clock cycle long. We test the attack by setting the `repeat` parameter of the glitch controller to 2 and 3.

In both cases, we notice a higher probability of success with an offset of 1 and a positive width, or a width of -1 and a positive offset.

(a) `repeat=2`



(b) `repeat=3`

Fig. 7: Glitch success frequency by glitch width and offset for fault model 3, with 10 tries per parameter pair.

References

1. Emmanuelle Anceaume, Yann Busnel, Ernst Schulte-Geers, and Bruno Sericola. Optimization results for a generalized coupon collector problem. *J. Appl. Probab.*, 53(2):622–629, 2016.
2. M. Baldi, A. Barenghi, L. Beckwith, J.-F. Biasse, T. Chou, A. Esser, K. Gaj, P. Karl, K. Mohajerani, G. Pelosi, E. Persichetti, M.-J. O. Saarinen, P. Santini, R. Wallace, and F. Zweydinger. LESS: Linear code equivalence signature scheme — round 2 specification. NIST PQC Round 2, 2025. https:

//csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/round-2/spec-files/less-spec-round2-web.pdf.
3. Marco Baldi, Alessandro Barenghi, Michele Battagliola, Sebastian Bitzer, Marco Gianvecchio, Patrick Karl, Felice Manganiello, Alessio Pavoni, Gerardo Pelosi, Paolo Santini, Jonas Schupp, Edoardo Signorini, Freeman Slaughter, Antonia Wachter-Zeh, and Violetta Weger. CROSS: Codes and restricted objects signature scheme — specification document, version 2.0. NIST PQC Round 2 submission, 2025.
4. Marco Baldi, Sebastian Bitzer, Alessio Pavoni, Paolo Santini, Antonia Wachter-Zeh, and Violetta Weger. Zero knowledge protocols and signatures from the restricted syndrome decoding problem. In *Public-Key Cryptography — PKC 2024, Part II*, volume 14603 of *Lecture Notes in Computer Science*, pages 243–274. Springer, 2024.
5. Carsten Baum, Ward Beullens, Lennart Braun, Cyprien Delpech de Saint Guilhem, Michael Klooß, Christian Majenz, Shibam Mukherjee, Emmanuela Orsini, Sebastian Ramacher, Christian Rechberger, Lawrence Roy, and Peter Scholl. FAEST v2: Algorithm Specifications. https://faest.info/faest-spec-v2.0.pdf, February 2025. Accessed: 2026-05-13.
6. Loïc Bidoux, Jesús-Javier Chi-Domínguez, Thibauld Feneuil, Philippe Gaborit, Antoine Joux, Matthieu Rivain, and Adrien Vinçotte. RYDE: a digital signature scheme based on rank syndrome decoding problem with MPC-in-the-Head paradigm. *Designs, Codes and Cryptography*, 93(5):1451–1486, May 2025.
7. Tung Chou, Ruben Niederhagen, Edoardo Persichetti, Tovohery Hajatiana Randrianarisoa, Krijn Reijnders, Simona Samardjiska, and Monika Trimoska. Take your MEDS: digital signatures from matrix code equivalence. In Nadia El Mrabet, Luca De Feo, and Sylvain Duquesne, editors, *Progress in Cryptology - AFRICACRYPT 2023 - 14th International Conference on Cryptology in Africa, Sousse, Tunisia, July 19-21, 2023, Proceedings*, Lecture Notes in Computer Science, pages 28–52. Springer, 2023.
8. O. Goldreich, S. Goldwasser, and S. Micali. How to construct random functions. *Journal of the ACM*, 33(4):792–807, 1986.
9. Xiao Huang, Zhuo Huang, Yituo He, Quan Yuan, Chao Sun, Mehdi Tibouchi, and Yu Yu. Faultless key recovery: Iteration-skip and loop-abort fault attacks on LESS. *IACR Cryptol. ePrint Arch.*, 2026:117, 2026.
10. Puja Mondal, Supriya Adhikary, Suparna Kundu, and Angshuman Karmakar. Zkfault: Fault attack analysis on zero-knowledge based post-quantum digital signature schemes. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part VIII*, Lecture Notes in Computer Science, pages 132–167. Springer, 2024.
11. Puja Mondal, Suparna Kundu, Hikaru Nishiyama, Supriya Adhikary, Daisuke Fujimoto, Yuichi Hayashi, and Angshuman Karmakar. Fault to forge: Fault assisted forging attacks on LESS signature scheme. *IACR Cryptol. ePrint Arch.*, 2025:1838, 2025.
12. Weiyu Xu and Ao Kevin Tang. A generalized coupon collector problem. *CoRR*, abs/1010.5608, 2010.

A Full Proofs

Throughout this appendix, $\mathcal{T}$ denotes a GGM seed tree with leaf set $\{0, \ldots, t-1\}$ and $\mathcal{F} : V(\mathcal{T}) \to \{0,1\}$ its flag labelling, as produced by the Publish phase of the relevant GZKST instance. We use the convention $\mathcal{F}(v) = 1$ to mean that v's subtree contains at least one hidden leaf ($i \in \mathcal{H}$), and $\mathcal{F}(v) = 0$ to mean that all leaf descendants of v are revealed ($i \in \mathcal{R}$).

A.1 Proof of Proposition 1 (LESS-v2 is a GZKST instance)

Let $\mathcal{T}$ be the LESS-v2 seed tree with $\lceil \log_2 t \rceil$ levels. After the three-phase flag-tree construction the flag values are:

$$\mathcal{F}(\text{leaf}_i) = \mathbf{1}[i \in \mathcal{H}], \quad i \in \{0, \ldots, t-1\}, \tag{1}$$

$$\mathcal{F}(v) = \mathcal{F}(\ell) \vee \mathcal{F}(r), \quad \text{for every internal node } v \text{ with children } \ell, r. \tag{2}$$

Equations (1)–(2) jointly imply:

$$\mathcal{F}(v) = 1 \iff \exists\, i \in \mathcal{H} : \text{leaf}_i \text{ is a descendant of } v. \tag{3}$$

The published path is $\mathsf{path} = \{v : \mathcal{F}(v) = 0,\ \mathcal{F}(\text{parent}(v)) = 1\}$ (with the convention that $\mathcal{F}(\text{parent}(\text{root})) = 1$).

Proof (Proof of Invariant 1 (Correctness)). Let $i \in \mathcal{R}$. Consider the root-to-leaf path $\pi_i = (v_0 = \text{root}, v_1, \ldots, v_d = \text{leaf}_i)$.

Since $|\mathcal{H}| \geq 1$ (the challenge weight satisfies $w \geq 1$), at least one leaf has flag 1. By (3), $\mathcal{F}(v_0) = \mathcal{F}(\text{root}) = 1$.

Since $i \in \mathcal{R}$, equation (1) gives $\mathcal{F}(v_d) = 0$.

Because $\mathcal{F}(v_0) = 1$ and $\mathcal{F}(v_d) = 0$, the path π_i contains at least two nodes with different flags. Define $k = \min\{j \in \{0, \ldots, d\} : \mathcal{F}(v_j) = 0\}$. Then $k \geq 1$ (since $\mathcal{F}(v_0) = 1$), $\mathcal{F}(v_k) = 0$, and $\mathcal{F}(v_{k-1}) = 1$. Hence $v_k \in \mathsf{path}$, and v_k lies on the root-to-leaf path π_i, establishing Invariant 1.

Proof (Proof of Invariant 2 (ZK hiding)). Let $i \in \mathcal{H}$ and let v be any proper ancestor of leaf_i (including the root). Since leaf_i is a descendant of v and $i \in \mathcal{H}$, equation (3) gives $\mathcal{F}(v) = 1$. Hence no ancestor of leaf_i can satisfy $\mathcal{F}(v) = 0$, so no ancestor of leaf_i lies in path. Invariant 2 holds.

Proof (Extension to incomplete trees (Remark 1)). When t is not a power of 2, the LESS-v2 tree is incomplete: some leaves appear at depth $\lceil \log_2 t \rceil - 1$ rather than $\lceil \log_2 t \rceil$. The index-tracking arrays npl, ll, off, and lsi ensure that $\mathsf{LabelLeaves}$ correctly identifies which nodes are leaves and assigns flags according to (1), and that $\mathsf{ComputeSeeds}$ performs the bottom-up OR pass (2) over the actual tree structure.

The proofs of both invariants above are purely graph-theoretic and depend only on the properties (1)–(3), which hold regardless of whether leaves appear at a uniform depth. Hence both invariants hold for any incomplete tree shape produced by the LESS-v2 expansion.

A.2 Proof of Proposition 2 (MEDS is a GZKST instance)

$\mathsf{SeedTreeToPath}$ implements the following erasure: for each $i \in \mathcal{H}$ the algorithm (i) removes the label of leaf_i, and (ii) propagates the removal upward—a node

v loses its label as soon as *at least one* leaf in its subtree has been removed. By induction this is equivalent to the OR-propagation $\mathcal{F}(v) = \mathcal{F}(\ell) \vee \mathcal{F}(r)$ of LESS-v2.

Claim. After the erasure pass, a node v retains its label if and only if $\mathcal{F}(v) = 0$ in the sense of (3), i.e., *every* leaf descendant of v is in $\mathcal{R}$ (no hidden leaf in v's subtree).

Proof. ($\Rightarrow$) Suppose v is retained. By definition of the upward propagation, no ancestor of any hidden leaf is retained. Hence v's subtree contains no hidden leaf, i.e., $\mathcal{F}(v) = 0$.

($\Leftarrow$) Suppose $\mathcal{F}(v) = 0$, i.e., every leaf in v's subtree is in $\mathcal{R}$. The erasure step removes only hidden leaves and their ancestors. Since v's subtree contains no hidden leaf, no removal is triggered in v's subtree, so v retains its label.

Remark 7. The claim corrects a previous version of this proof that stated "a node retains its label iff its subtree contains at least one revealed leaf." The correct condition is stronger: *all* leaves must be revealed ($\mathcal{F}(v) = 0$), which is consistent with the LESS-v2 construction and with the actual SEEDTREETOPATH code.

The published path path consists of the highest surviving nodes in the tree, serialised in depth-first order; equivalently, $\mathsf{path} = \{v : v \text{ retains its label}, \ \mathrm{parent}(v) \text{ was erased}\}$. By the claim, this is $\{v : \mathcal{F}(v) = 0, \ \mathcal{F}(\mathrm{parent}(v)) = 1\}$, which is identical to the LESS-v2 definition.

Invariants 1 and 2 follow by the same argument as in Appendix A.1.

A.3 Proof of Proposition 3 (CROSS is a GZKST instance)

The seed-tree component of CROSS (SeedTreePath) is structurally equivalent to MEDS's SeedTreeToPath (see Section 3.2). The argument of Appendix A.2 therefore applies directly, establishing Invariants 1 and 2 for the seed-hiding part.

The additional Merkle proof over $\{\mathsf{cmt}_0[i]\}_{i \in \mathcal{H}}$ is an authenticity mechanism for the commitment digests that the verifier cannot recompute from the revealed seeds. It neither publishes any seed in $\{\widetilde{s}_i : i \in \mathcal{H}\}$ nor affects the seed-path computation, and is therefore orthogonal to both invariants.

A.4 Proof of Proposition 4 (Correctness of Algorithm 4)

Let $m = |\mathcal{S}| = s - 1$. We model collection of secret components as a *generalised coupon-collector* problem [12,1] with m distinct coupons.

Correctness of Extract. For each scheme, Definition 5 specifies Extract as a closed-form algebraic procedure: matrix inversion over $\mathbb{F}_q$ (MEDS), column extraction (LESS), or a group operation (CROSS). Each procedure is deterministic and correct given the dual revelation $(\widetilde{s}_i, \mathsf{resp}_i)$: the algebraic relationship between the response and the hidden seed is an identity (not an approximation), so Extract returns exactly $\mathsf{sk}' =$ the secret component indexed by $j = c_i$.

Termination. We work under the assumption of Proposition 4: $v^* = \mathrm{root}$, so the subtree of v^* is the entire tree and $L = |\mathcal{H} \cap \mathrm{leaves}(\mathrm{root})| = |\mathcal{H}| = w$ exactly—a deterministic quantity, not a random variable. Define

$$p_{\min} \;=\; 1 - \left(\frac{s-2}{s-1}\right)^{w} \;=\; 1 - \left(1 - \frac{1}{s-1}\right)^{w}.$$

For any $r \geq 1$ uncollected secrets, the probability that at least one of the w hidden leaves reveals a new secret index is

$$p_r \;=\; 1 - \left(1 - \frac{r}{s-1}\right)^{w} \;\geq\; 1 - \left(1 - \frac{1}{s-1}\right)^{w} \;=\; p_{\min} \;>\; 0,$$

where the inequality holds because $1 - r/(s-1)$ is decreasing in r, so $r = 1$ gives the weakest bound. Hence

$$\Pr\big[\mathsf{new}(q) \geq 1 \;\big|\; \mathsf{Collected}\big] \;\geq\; p_{\min} \;>\; 0 \qquad \text{for all } s \geq 2,\; w \geq 1.$$

Let T_j be the number of effective queries to go from $|\mathsf{Collected}| = j-1$ to $|\mathsf{Collected}| = j$. Conditional on $|\mathsf{Collected}| = j-1$, each query succeeds with probability at least $p_{\min}$, so T_j is stochastically dominated by a $\mathrm{Geom}(p_{\min})$ random variable, which is finite almost surely. Hence $Q = \sum_{j=1}^{m} T_j$ is finite almost surely, and Algorithm 4 terminates with probability 1.

Expected query count and tail bound. Let R_k be the number of uncollected secret indices after k effective queries, and let $r = R_{k-1} > 0$. Each of the w hidden leaves (recall $v^* = \mathrm{root}$, so all w hidden leaves are exposed) independently draws a uniform index from $\{1, \ldots, s-1\}$, so the probability that *none* of the w draws falls in the remaining set of size r is $\big((s-1-r)/(s-1)\big)^{w}$. Hence the probability of revealing at least one new index is

$$p_r \;=\; 1 - \left(\frac{s-1-r}{s-1}\right)^{w} \;=\; 1 - \left(1 - \frac{r}{s-1}\right)^{w}. \tag{4}$$

Since $1 - r/(s-1)$ is decreasing in r, we have $p_r \geq p_{\min} = 1 - (1 - 1/(s-1))^{w}$ for all $r \geq 1$.

The expected total number of effective queries is

$$\mathbb{E}[Q] \;=\; \sum_{r=1}^{m} \frac{1}{p_r} \;\leq\; \frac{m}{p_{\min}} \;=\; \frac{s-1}{p_{\min}}, \tag{5}$$

where the inequality uses $p_r \geq p_{\min}$ for all $r \geq 1$.

For the tail bound, each $T_j \leq k_0$ with probability at least $1-(1-p_{\min})^{k_0}$ (geometric tail), so by a union bound over $m = s-1$ stages,

$$\Pr[Q > k] \;\leq\; (s-1)\cdot(1-p_{\min})^{\lfloor k/(s-1)\rfloor}.$$

Setting $(s-1)(1-p_{\min})^{\lfloor k/(s-1)\rfloor} \leq \delta$ and solving gives $k \leq \left\lceil \frac{s-1}{p_{\min}} \cdot \ln \frac{s-1}{\delta} \right\rceil$, establishing the bound in Proposition 4.

Output correctness. When the loop terminates, $|\mathsf{Collected}| = m$, and for each $j \in \{1,\dots,s-1\}$ there is exactly one entry $(j, \mathsf{sk}'_j) \in \mathsf{Collected}$ with $\mathsf{sk}'_j = A_j^{-1}$ (or the corresponding secret for each scheme). The Collect step assembles these into $\widehat{\mathsf{sk}}$, which equals sk by correctness of Extract.

A.5 Derivation of Proposition 5 (Expected query count)

Setup. We work in the root-fault setting of Proposition 5: $v^* = \mathrm{root}$, so $\ell = t$ and $L = w$ exactly (deterministic). By the Fiat–Shamir uniformity assumption, the challenge index c_i is uniform in $\{1,\dots,s-1\}$ independently for each $i \in \mathcal{H}$.

Distinct secrets per query. Conditioning on $L = w$, the number of *distinct* secret indices revealed in one effective query has expectation

$$\mathbb{E}[\#\text{distinct} \mid L = w] = (s-1)\left(1 - \left(1 - \frac{1}{s-1}\right)^{w}\right) \;=\; E_{\mathrm{dist}}, \tag{6}$$

by linearity of expectation and symmetry of the uniform distribution.

Jensen remark for general fault location. When $v^* \neq \mathrm{root}$ and L is a hypergeometric random variable with mean $\bar{w} = w\ell/t$, taking expectation over L gives

$$\mathbb{E}[\#\text{distinct}] = (s-1)\left(1 - \mathbb{E}\left[\left(1 - \tfrac{1}{s-1}\right)^{L}\right]\right).$$

Since $f(x) = \alpha^x$ with $\alpha = 1 - 1/(s-1) \in (0,1)$ is convex, Jensen's inequality gives $\mathbb{E}[f(L)] \geq f(\bar{w}) = \alpha^{\bar{w}}$, hence

$$\mathbb{E}[\#\text{distinct}] \;\leq\; (s-1)\left(1 - \left(1 - \frac{1}{s-1}\right)^{\bar{w}}\right) \;=:\; E^{\bar{w}}_{\mathrm{dist}}. \tag{7}$$

$E^{\bar{w}}_{\mathrm{dist}}$ is an *upper* bound on expected per-query gain, which implies a *lower* bound on $\mathbb{E}[Q]$. It cannot be used to derive an upper bound on $\mathbb{E}[Q]$; the upper bound in Proposition 5 follows solely from the direct inequality $p_r \geq p_{\min}$ established in the root-fault setting.

Total queries (root fault). From equation (4) in Appendix A.5, $p_r = 1 - \big(1 - r/(s-1)\big)^w$ (with $L = w$ deterministic), and since $p_r \geq p_{\min}$ for all $r \geq 1$,

$$\mathbb{E}[Q] = \sum_{r=1}^{m} \frac{1}{p_r} \leq \frac{m}{p_{\min}} = \frac{s-1}{p_{\min}},$$

which for $p_{\min} \approx 1$ (all MEDS parameter sets, cf. Table 2) gives $\mathbb{E}[Q] \lesssim s - 1$.